



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO
COMPUTADORA

EJERCICIOS DE CLASE N° 01

NOMBRE COMPLETO: Cruz Cano Omar

N° de Cuenta: 318068932

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 5 de Septiembre de 2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa.

1. Generar las figuras copiando los vértices de triángulo magenta y cuadrado azul: triángulo amarillo, triángulo verde, triángulo rojo, cuadrado café y cuadrado negro.

2. Usando la proyección ortogonal generar el siguiente dibujo a partir de instancias de las figuras anteriormente creadas, recordar que todos se dibujan en el origen y por transformaciones geométricas se desplazan. Los rombos azul y café son cuadrados rotados sobre el eje Z.

Primeramente, modificamos la línea que asigna el color del fondo para obtener uno distinto a blanco.

```
glClearColor(1.0f, 0.647f, 0.0f, 1.0f); // Naranja
```

Para este ejercicio haremos uso del shader para usar color como parte del VAO.

```
shaderList[1].useShader();  
uniformModel = shaderList[1].getModelLocation();  
uniformProjection = shaderList[1].getProjectLocation();
```

Para imprimir los tres triángulos apilados aproveché la posición del primer triángulo solamente para trasladarme en el eje Y hacia arriba y abajo para imprimir los 3 triángulos de diferentes colores. Es importante siempre actualizar la variable uniform antes de mandar a llamar las funciones Mesh o no se reflejarán los cambios.

```
// Tres triángulos apilados  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(-0.75f, 0.0f, -4.0f));  
model = glm::scale(model, glm::vec3(0.15f, 0.15f, 0.15f));  
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
meshColorList[4]->RenderMeshColor();  
  
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, -1.0f));  
model = glm::translate(model, glm::vec3(0.0f, -2.0f, 0.0f));  
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
meshColorList[3]->RenderMeshColor();  
  
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, -1.0f));  
model = glm::translate(model, glm::vec3(0.0f, 4.0f, 0.0f));  
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
meshColorList[2]->RenderMeshColor();
```

Para los rectángulos cafés ocupé el mismo concepto de aprovechar la impresión del primero para imprimir el segundo después de una traslación sobre el eje X.

```
// Rectangulos cafres
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-0.925f, 0.0f, -3.9f));
model = glm::scale(model, glm::vec3(0.05f, 1.3f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[6]->RenderMeshColor();

model = glm::translate(model, glm::vec3(7.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[6]->RenderMeshColor();
```

El rectángulo negro fue el más sencillo de implementar ya después de saber dónde terminarían los rectángulos cafés solo fue necesario agregar una figura al arreglo que corresponde a un cuadrado negro para posteriormente escalarlo en X y trasladarlo en Y.

```
// Rectangulo negro
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -0.675f, -3.9f));
model = glm::scale(model, glm::vec3(1.95f, 0.05f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[7]->RenderMeshColor();
```

Para los triángulos apilados de tal forma que formaban un triángulo empecé por una esquina y a través de traslaciones y rotaciones fui terminando la figura. Es muy importante considerar que después de escalar y rotar y mostrar una figura, los ejes de tal figura se modifican, por lo que se recomienda aplicar una transformación inversa para trabajar con los ejes X, Y y Z normales.

```
// 4 Triangulos en triangulo
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.8f, -0.5488f, -4.0f));
model = glm::scale(model, glm::vec3(0.10f, 0.10f, 0.10f));
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[4]->RenderMeshColor();

model = glm::translate(model, glm::vec3(-1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[2]->RenderMeshColor();

model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, -1.0f));
model = glm::translate(model, glm::vec3(-1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[3]->RenderMeshColor();

model = glm::translate(model, glm::vec3(1.0f, 2.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[1]->RenderMeshColor();
```

Por último, para la figura central imprimí primero los cuadrados rotados en el eje z para asemejar un rombo y después implementé cada uno de los triángulos a través de rotaciones y traslaciones, para lograr que toda la figura formara un cuadrado fue necesario aplicar una escala en el eje Y al triángulo antes de imprimirlo. Como se mencionó anteriormente, fue de vital importancia aplicar una transformación inversa después de rotar o escalar nuestro modelo para poder trabajar sobre los ejes X y Y y que las rotaciones no afecten los ejes.

```
//Figura central
model = glm::mat4(1.0);
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
model = glm::translate(model, glm::vec3(0.0f, -0.92f, -4.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[5]->RenderMeshColor();

model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.1f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[6]->RenderMeshColor();

model = glm::scale(model, glm::vec3(2.0f, 2.0f, 1.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, -1.0f));
model = glm::translate(model, glm::vec3(0.53f, 0.53f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, -1.0f));
model = glm::scale(model, glm::vec3(1.0f, 0.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[4]->RenderMeshColor();

model = glm::scale(model, glm::vec3(1.0f, 2.0f, 1.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-2.135f, 0.0f, 0.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(1.0f, 0.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[2]->RenderMeshColor();

model = glm::scale(model, glm::vec3(1.0f, 2.0f, 1.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, -1.0f));
model = glm::translate(model, glm::vec3(0.0f, -2.1f, 0.0f));
model = glm::rotate(model, 135 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(1.0f, 0.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[1]->RenderMeshColor();

model = glm::scale(model, glm::vec3(1.0f, 2.0f, 1.0f));
model = glm::rotate(model, 135 * toRadians, glm::vec3(0.0f, 0.0f, -1.0f));
model = glm::translate(model, glm::vec3(2.135f, 0.0f, 0.0f));
model = glm::rotate(model, 225 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(1.0f, 0.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[3]->RenderMeshColor();
```

2. Conclusión:

Esta práctica permitió comprender de forma práctica el uso de proyecciones los dos tipos de proyecciones revisados al momento, shaders con VAO y matrices de transformación. La mayor complejidad estuvo en manejar el orden de traslaciones, rotaciones y escalas sin desfasar los ejes locales de las figuras, lo cual se resolvió aplicando transformaciones inversas y actualizando correctamente las variables uniform antes del renderizado.

Finalmente, esta práctica resulta fundamental ya que sienta las bases técnicas y conceptuales para el desarrollo de las sesiones posteriores; dominar el manejo de shaders, la actualización de variables uniform y la manipulación precisa de matrices de transformación no solo permite construir escenas bidimensionales ordenadas, sino que es el punto de partida indispensable para abordar con éxito temas más avanzados como transformaciones jerárquicas, proyecciones en tres dimensiones, iluminación y cámaras virtuales.